

# A simple ALU (“arithmetic logic unit”)

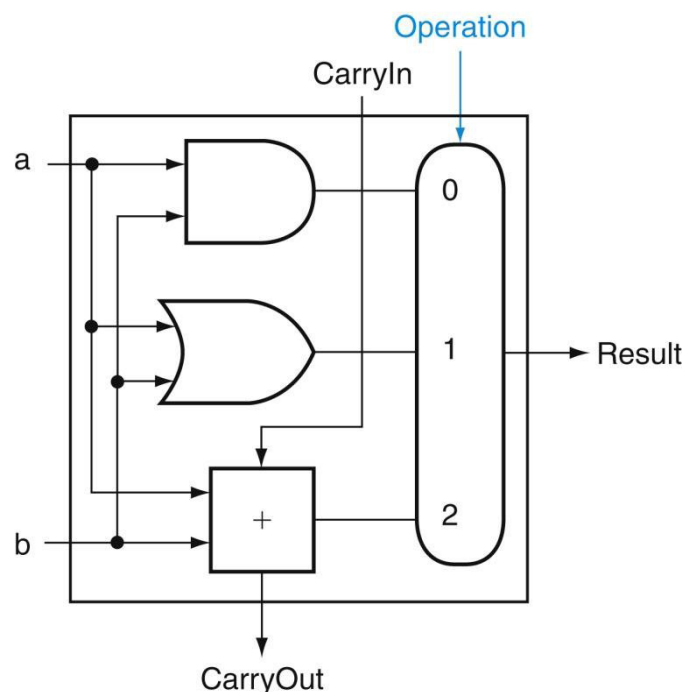
(reference: text Sections A.5, A.6)

initially, will design a simple ALU that implements only the RISC-V “add”, “and”, and “or” instructions

two key ideas in design:

- do **all** operations in parallel, use **multiplexor** to select the desired result
- build 32-bit ALU by using **1-bit ALUs** as building blocks

**1-bit ALU**: when used in our 32-bit ALU, the  $a$ ,  $b$  inputs to the  $i$ 'th 1-bit ALU will be the  $i$ 'th bits of the two 32-bit operands (from text Fig. A.5.6):



1-bit ALU uses 1-bit full adder, together with AND gate, OR gate, and multiplexor

## 1-bit full adder

start with truth table:

| <i>A</i> | <i>B</i> | <i>CarryIn</i> |  | <i>CarryOut</i> | <i>Sum</i> |
|----------|----------|----------------|--|-----------------|------------|
| 0        | 0        | 0              |  | 0               | 0          |
| 0        | 0        | 1              |  | 0               | 1          |
| 0        | 1        | 0              |  | 0               | 1          |
| 0        | 1        | 1              |  | 1               | 0          |
| 1        | 0        | 0              |  | 0               | 1          |
| 1        | 0        | 1              |  | 1               | 0          |
| 1        | 1        | 0              |  | 1               | 0          |
| 1        | 1        | 1              |  | 1               | 1          |

consider first the *CarryOut* output:

$$\begin{aligned}
 \text{CarryOut} &= \bar{A} \cdot B \cdot \text{CarryIn} + A \cdot \bar{B} \cdot \text{CarryIn} \\
 &\quad + A \cdot B \cdot \overline{\text{CarryIn}} + A \cdot B \cdot \text{CarryIn} \\
 &= \bar{A} \cdot B \cdot \text{CarryIn} + A \cdot \bar{B} \cdot \text{CarryIn} + A \cdot B \cdot \overline{\text{CarryIn}} \\
 &\quad + A \cdot B \cdot \text{CarryIn} + A \cdot B \cdot \text{CarryIn} + A \cdot B \cdot \text{CarryIn} \\
 &= (\bar{A} + A) \cdot B \cdot \text{CarryIn} + (\bar{B} + B) \cdot A \cdot \text{CarryIn} \\
 &\quad + (\overline{\text{CarryIn}} + \text{CarryIn}) \cdot A \cdot B \\
 &= B \cdot \text{CarryIn} + A \cdot \text{CarryIn} + A \cdot B
 \end{aligned}$$

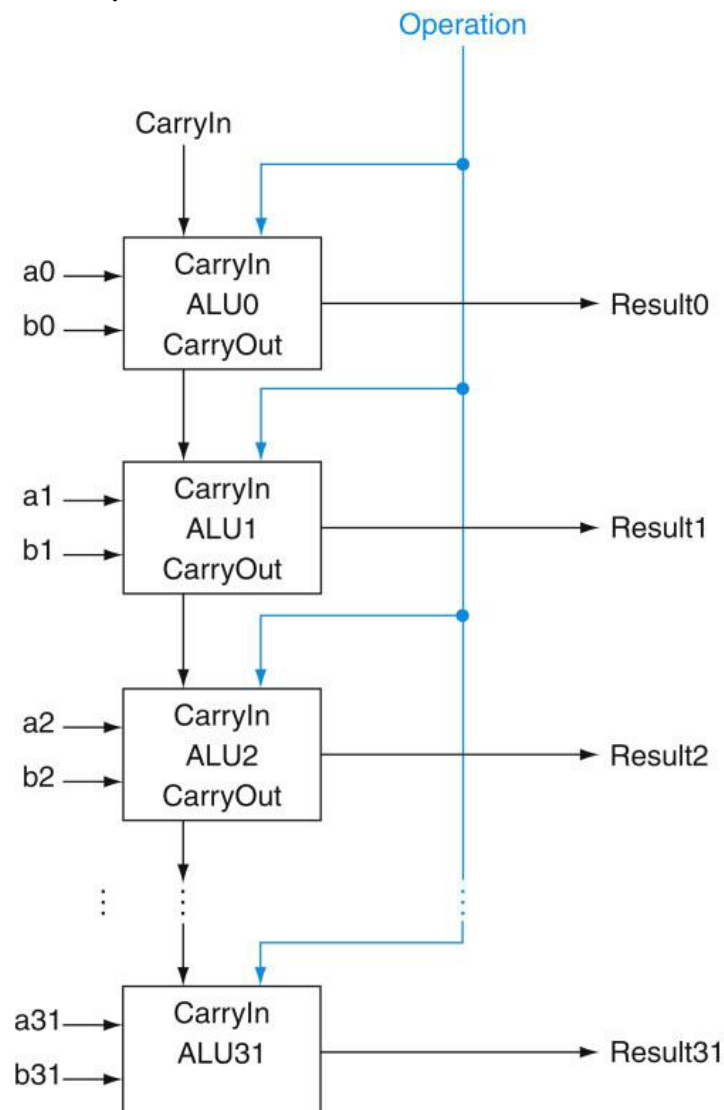
how about the *Sum* output:

$$\begin{aligned} Sum = & \overline{A} \cdot \overline{B} \cdot CarryIn + \overline{A} \cdot B \cdot \overline{CarryIn} \\ & + A \cdot \overline{B} \cdot \overline{CarryIn} + A \cdot B \cdot CarryIn \end{aligned}$$

can this be simplified?

## 32-bit ALU using “ripple carry” design

(from text Fig. A.5.7)

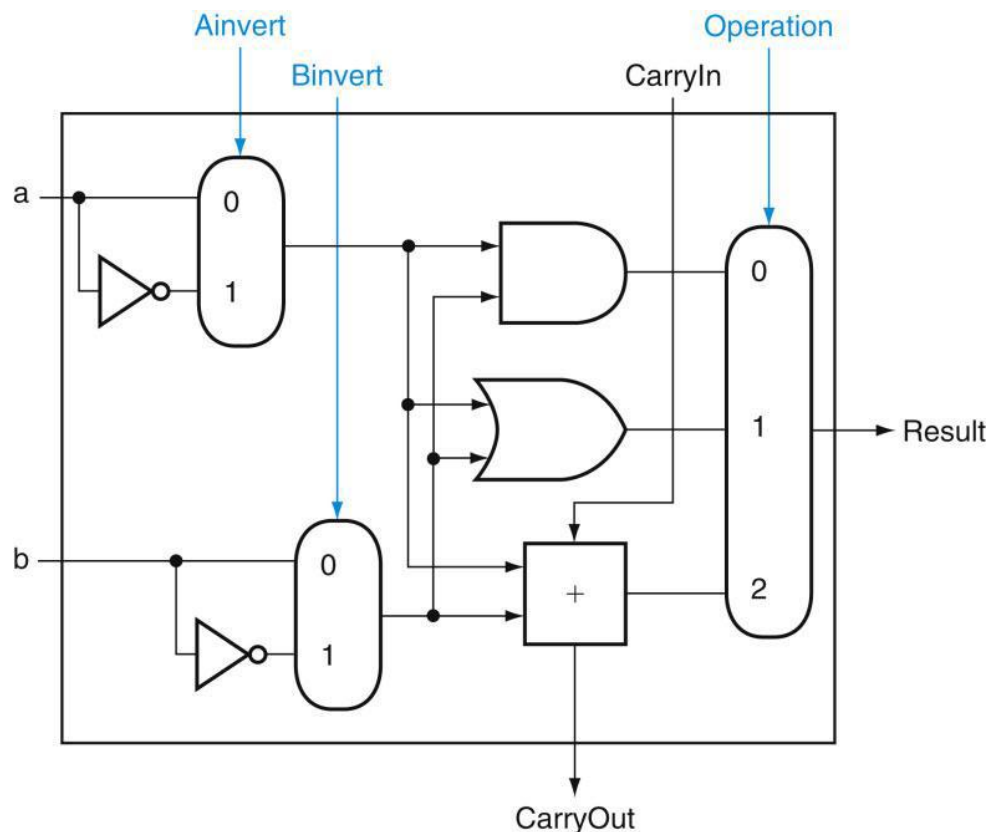


## two enhancements to our 32-bit ALU design

### subtraction

- implement  $a - b$  as  $a + (-b)$
- find  $-b$  by taking **bit complement** of  $b$  and **adding one**
- need to modify the 1-bit ALUs for taking bit complement; **do the “adding one” by setting  $CarryIn_0$  to 1** when do the addition with  $a$

below design (from text Fig. A.5.9) also includes ability to take bit complement of  $a$  (e.g. could be used to implement a “nor” instruction, since  $\overline{A + B} = \overline{A} \cdot \overline{B}$ )



## ALU control lines and corresponding operations (from text Fig. A.5.13)

| ALU control lines | Function |
|-------------------|----------|
| 0000              | AND      |
| 0001              | OR       |
| 0010              | add      |
| 0110              | subtract |

leftmost control line in table is **Ainvert**, then comes **Bnegate** (combining CarryIn0 and **Binvert**), and finally the two **Operation** bits

### “beq” / “bne” instructions

- could implement by subtracting the two operands and forming the “nor” of all the result bits
- this yields an output from the ALU called the “**zero**” output, which is 1 if and only if the result of the ALU operation is zero
- the “zero” output then used by **control** part of the processor

but how good is this ALU design?

- “ripple-carry” addition is *slow* ... takes  $\Theta(n)$  time to add two  $n$ -bit numbers
- **carry-lookahead addition** is one of several faster approaches that are used in practice

### carry-lookahead addition

define  $g_k = a_k \cdot b_k$  and  $p_k = a_k + b_k$

if  $g_k$  is 1, then we know that a carry out will be **generated** when adding the bits at position  $k$

if  $p_k$  is 1, then we know that any carry out from adding the bits at position  $k-1$  will be **propagated**

and so we can write down the following logic equation for the **carry in** at each bit position  $i$  ...

$$\text{CarryIn}_i = g_{i-1} + g_{i-2} \cdot p_{i-1} + g_{i-3} \cdot p_{i-2} \cdot p_{i-1} + \dots + g_0 \cdot p_1 \cdot p_2 \cdot \dots \cdot p_{i-1} + \text{CarryIn}_0 \cdot p_0 \cdot p_1 \cdot \dots \cdot p_{i-1}$$

wow, 5 step addition! – (1) evaluate all  $g_k, p_k$  (in parallel), (2) evaluate all the product terms needed for all  $\text{CarryIn}_i$  (in parallel), (3) evaluate all  $\text{CarryIn}_i$  (in parallel), (4 and 5) use 1-bit ALUs to get all result bits (in parallel)

but ... this naive “5 step” approach would require using gates (in steps 2 and 3) with “fan-in” (number of inputs) that grows linearly with  $n$  (!)